Universidad de Buenos Aires
Facultad de Ingeniería
Sistemas Distribuidos I (75.74)

# Trabajo Práctico Final

## Diseño del sistema de análisis de artículos de Wikipedia

2.º cuatrimestre de 2026

Integrantes

| Nombre | Padrón |
|---|---|
| Mateo Castaño | 110114 |
| Cristobal Alvarez | 110308 |
| Martin Palazon | 102679 |

# Índice


Diagramas C4

1. Contexto
2. Contenedores
3. Q2 en detalle
4. Q3 y Q4 en detalle
5. Q5 en detalle

DAG del sistema

1. Vista general
2. Entrada: cliente a colas bio y texto
3. Q1 y Q2
4. Text processor: de la cola texto a palabras, promedio y comparación
5. Q3 y Q4: contador de palabras y joins
6. Q5: promedio y comparador
7. Salida: cola de resultados a cliente

Diagramas de robustez

1. Entrada, salida y Q1
2. Q2
3. Q3 y Q4
4. Q5

Diagramas de actividades

1. Q2
2. Q3 y Q4
3. Q5

Diagramas de secuencia

1. Q1
2. Q2
3. Q3
4. Q4
5. Q5

Diagramas de paquetes

1. Vista general
2. Paquete nodes
3. Paquetes middleware y common

Diagramas de despliegue

# Diagramas C4

Usamos tres niveles del modelo C4. El nivel 1 muestra el contexto del sistema, el nivel 2 sus contenedores, con los nodos de cada consulta agrupados, y el nivel 3 abre cada grupo para mostrar sus nodos y lo que consume y publica cada uno.

# 1. Contexto

El Cliente es una aplicación externa: lee el dataset Wikipedia Structured Contents (archivos Parquet de Kaggle), le manda al sistema los artículos en batches y recibe los resultados de las 5 consultas. El sistema atiende a varios clientes a la vez y depende de un solo sistema externo, Wikimedia (upload.wikimedia.org), de donde descarga las imágenes de Q2.

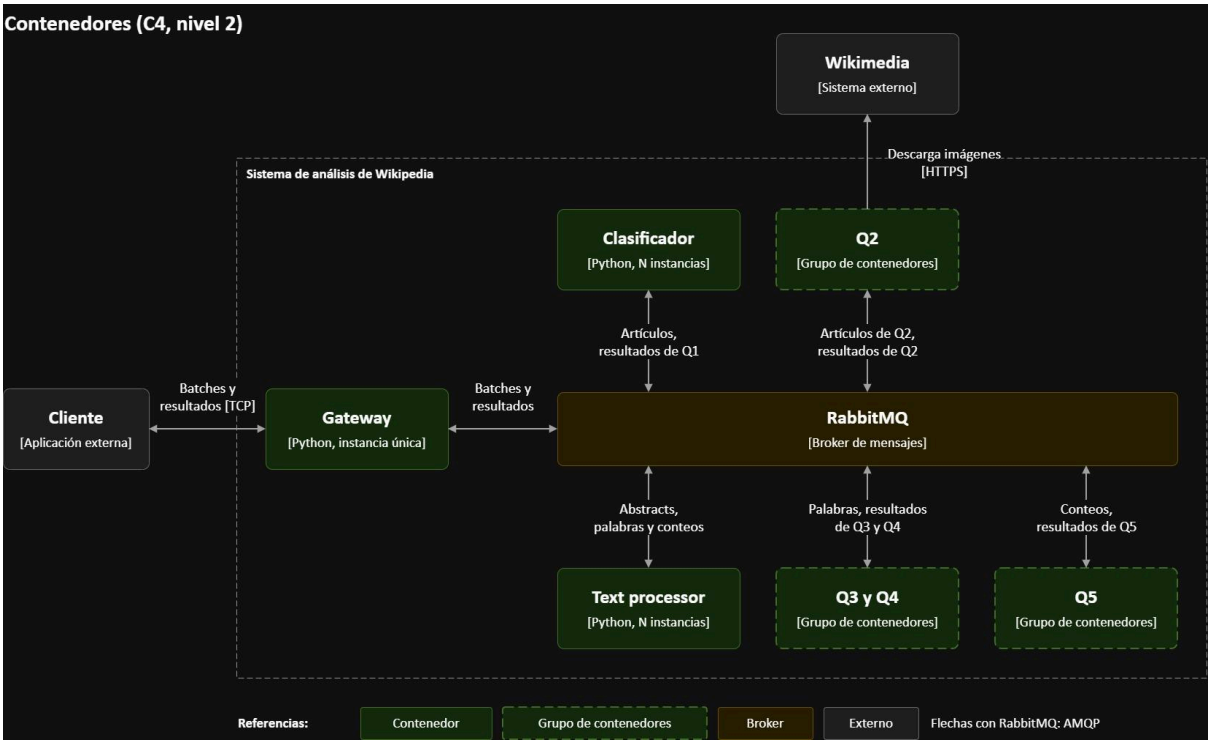


## 2. Contenedores

Muestra las piezas que se despliegan por separado. Para que se lea mejor, los nodos propios de Q2, de Q3, Q4 y de Q5 se agrupan en una caja punteada por consulta, que se abre en el nivel 3.

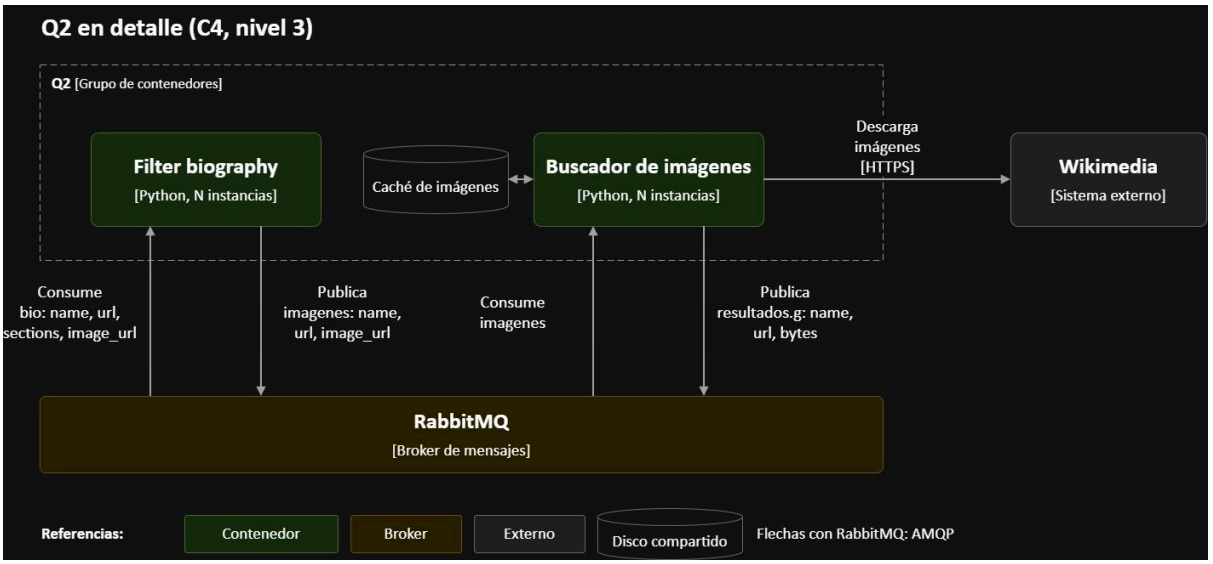
El Cliente se conecta por TCP a un único Gateway y usa esa conexión tanto para mandar los batches como para recibir los resultados. Los contenedores no se comunican directamente entre sí: todos los mensajes pasan por RabbitMQ. Los datos se publican en colas durables por el exchange por defecto, y cuatro exchanges direct de control reparten los EOF entre las réplicas de cada grupo. Clasificador, Filter biography, Buscador de imágenes y Text processor comparten una cola por grupo, y los demás nodos tienen una cola por instancia, elegida por el productor. El Clasificador resuelve Q1 y reparte los artículos, y el Text processor es la etapa común de Q3, Q4 y Q5, por eso quedan fuera de los grupos.

El grupo de Q2 es el único que habla con un sistema externo: descarga las imágenes de Wikimedia.



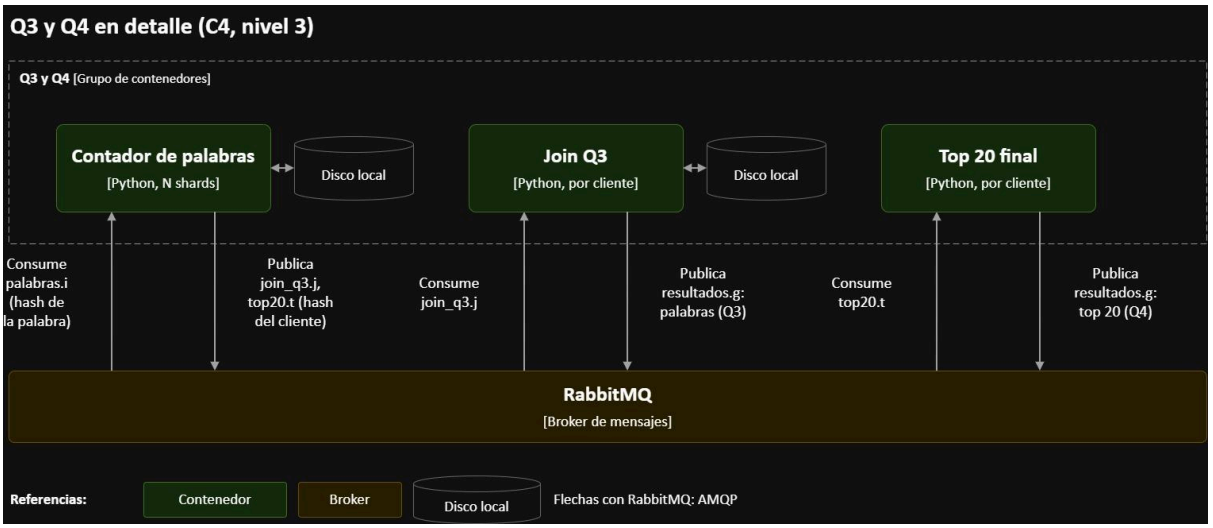
## 3. Q2 en detalle

El Filter biography consume bio y publica en imagenes los artículos que mencionan biography o biographie. El Buscador de imágenes consume imagenes, busca cada imagen en una caché en disco que comparten todas las réplicas, la descarga de Wikimedia si no está y publica el resultado de Q2 en resultados.g.



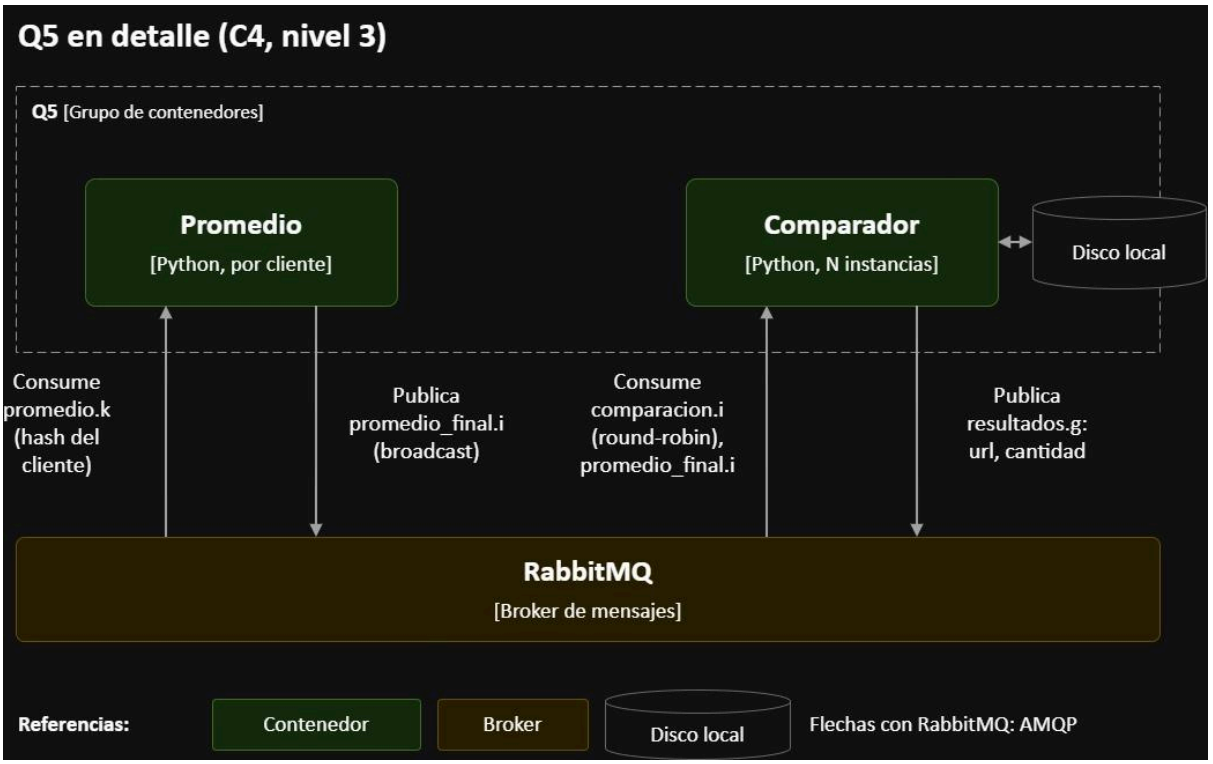
## 4. Q3 y Q4 en detalle

El Contador de palabras consume palabras.i, repartida por hash de la palabra, y guarda las entradas en disco hasta el EOF del cliente. Después publica en join_q3.j y en top20.t, repartidas por hash del cliente. El Join Q3 guarda en disco las listas de todos los contadores y publica el resultado de Q3, y el Top 20 final junta los tops locales y publica el de Q4. Los dos publican en resultados.g.



## 5. Q5 en detalle

El Promedio consume promedio.k, repartida por hash del cliente, y publica en promedio_final.i, una cola por comparador, para que les llegue a todos. El Comparador consume su cola comparacion.i, repartida por round-robin, y su cola promedio_final.i, guarda en disco los artículos hasta que llega el promedio y publica en resultados.g los que lo superan.



# DAG del sistema

El sistema se presenta en 7 diagramas de flujo de datos, de izquierda a derecha, con pocas entidades cada uno para poder leerlos claramente.

Los nodos con estado guardan sus datos por cliente: Contador de palabras, Join Q3 y Comparador en disco, y Top 20 final y Promedio en memoria. Los EOF viajan por las mismas colas que los datos, y entre las réplicas de un grupo con cola compartida van por un exchange de control. Ninguna de las dos cosas se dibuja, para no mezclar dimensiones. Clasificador, Filter biography, Buscador de imágenes y Text processor comparten una cola por grupo, y RabbitMQ le da cada mensaje a la instancia que esté libre. Los demás nodos tienen una cola por instancia, y el productor elige a cuál mandar por hash o por round-robin.

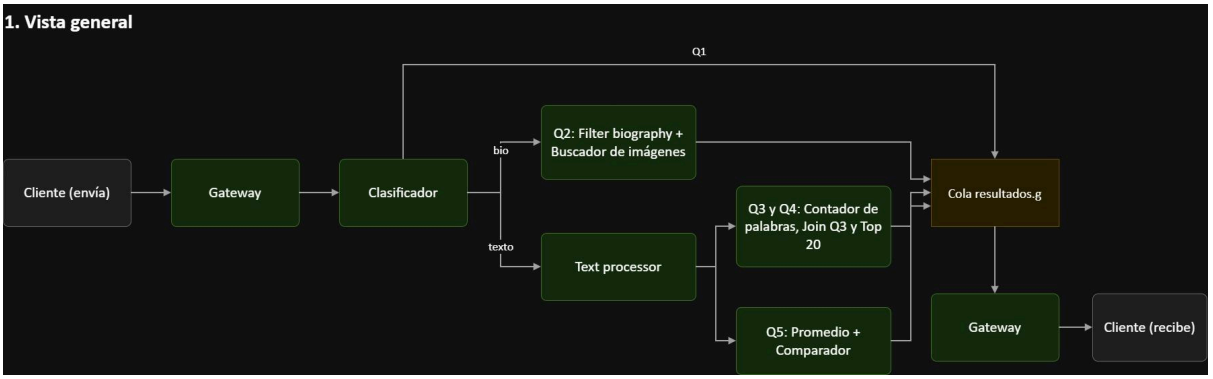
Referencias de colores:

| Nodo de cómputo | Cola | Cliente / entidad externa |
|---|---|---|

# 1. Vista general

Muestra el recorrido completo de un pedido, desde que el cliente envía los batches hasta que recibe los resultados.

El Clasificador separa cada batch en dos tipos de mensaje: bio (Q2) y texto (Q3, Q4 y Q5). Q1 se resuelve en el propio clasificador, que publica su resultado directo en la cola de resultados. Todos los flujos terminan en esa cola, resultados.g, y el gateway reenvía cada resultado al cliente a medida que sale.



## 2. Entrada: cliente a colas bio y texto

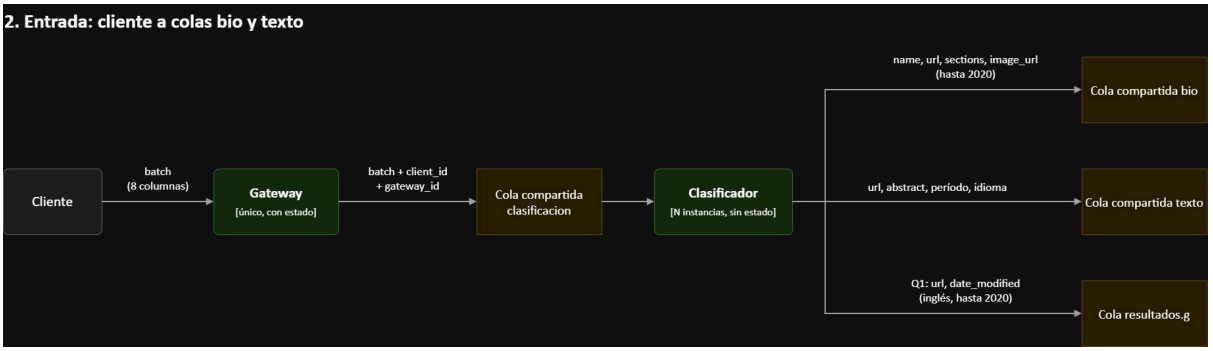
El cliente lee el archivo Parquet, arma batches con las 8 columnas necesarias y se los manda al gateway. El Gateway (único, con estado) se queda con esas columnas, agrega client_id y gateway_id y publica los batches en la cola clasificacion, que comparten todos los clasificadores.

Cada Clasificador (sin estado, N instancias) calcula el período (hasta_2020 o desde_2021) y el idioma de cada artículo, agrupa por destino y publica en dos colas:

**bio** (hasta 2020, con sections e image_url) va al Filter biography (Q2).

**texto** (todos los artículos con abstract) va al Text processor (Q3, Q4 y Q5). Las colas bio y texto las comparten todas las instancias de cada grupo.

Q1 no pasa por esas colas: para los artículos en inglés hasta 2020, el clasificador publica url y date_modified directo en la cola de resultados del gateway del cliente.



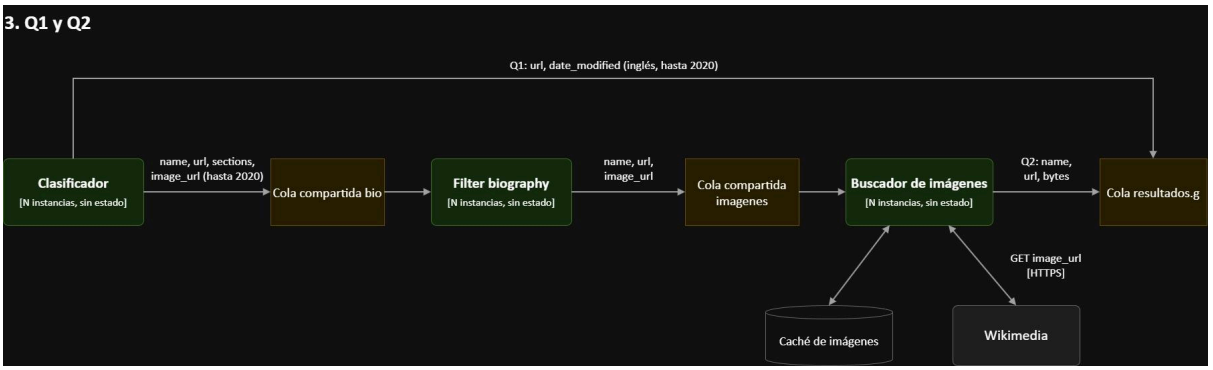
## 3. Q1 y Q2

Q1 no necesita nodos propios: el clasificador publica url y date_modified de los artículos en inglés hasta 2020 directo en la cola de resultados del gateway del cliente. Los resultados salen a medida que se clasifican los batches.

Q2 tiene dos etapas separadas porque la primera usa sobre todo CPU y la segunda espera a la red, así cada una escala por su lado.

El Filter biography (sin estado, N instancias) busca "biography" o "biographie" en sections, descarta sections y publica name, url e image_url en la cola imagenes, que comparten todos los buscadores.

El Buscador de imágenes (sin estado, N instancias) busca la imagen en la caché compartida y, si no está, la descarga desde upload.wikimedia.org. Reintenta sin límite los timeouts, errores de conexión, HTTP 429 y 5xx, y descarta las imágenes que fallan por otros motivos. Publica en la cola de resultados name, url y los bytes de la imagen.



## 4. Text processor: de la cola texto a palabras, promedio y comparación

Etapa común a Q3, Q4 y Q5, porque las tres cuentan las palabras de los abstracts sin contar las stopwords. El Text processor (sin estado, N instancias) toma batches de la cola texto, que comparten todos los Text processor, pasa cada abstract a minúsculas, separa las palabras, saca las stopwords de inglés y francés y cuenta las que quedan.

Antes de publicar agrupa todo el batch y lo manda directo a tres grupos de colas, cada uno repartido de forma distinta:

**palabras** (por hash de la palabra) llega al Contador de palabras i de Q3 y Q4.

**promedio** (suma de palabras distintas y cantidad de artículos hasta 2020, por hash de client_id) llega al Promedio k de Q5.

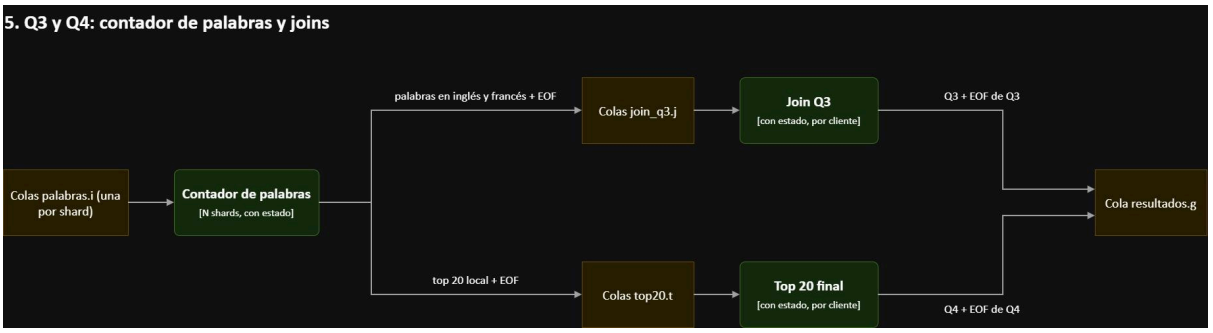
**comparación** (url y cantidad de palabras distintas de los artículos desde 2021, por round-robin) llega al Comparador i de Q5.



## 5. Q3 y Q4: contador de palabras y joins

Ambas consultas comparten el Contador de palabras (con estado, N shards). Como el hash de la palabra manda todas sus apariciones al mismo shard, cada shard tiene el conteo completo de sus palabras. El Contador guarda las entradas en disco, en varios archivos por cliente, hasta recibir los EOF de todos los Text processor y completar los conteos de mensajes; recién ahí emite lo de Q3 y Q4 y manda su EOF.

Para Q3 publica las palabras marcadas en inglés y en francés en join_q3.j; para Q4 publica su top 20 local en top20.t. Ambas colas se reparten por hash de client_id. El Join Q3 (con estado, por cliente) junta las listas de los N contadores, las publica en la cola de resultados y manda un único EOF de Q3. El Top 20 final (con estado, por cliente) ordena y publica las 20 primeras en la cola de resultados y manda un único EOF de Q4. Si hay empate en el puesto 20, desempata el orden alfabético.

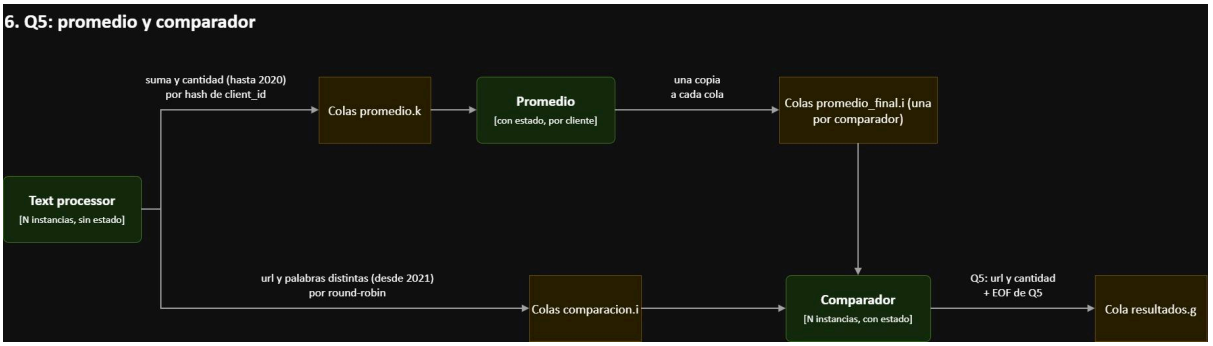


## 6. Q5: promedio y comparador

Q5 tiene dos ramas que trabajan en paralelo y se encuentran en el Comparador, porque ningún artículo desde 2021 se puede decidir hasta conocer el promedio de los artículos hasta 2020.

**Rama del promedio:** el Promedio (con estado, por cliente) acumula la suma de palabras distintas y la cantidad de artículos y, cuando terminan los datos del cliente, publica una copia de promedio_final en la cola promedio_final.i de cada Comparador.

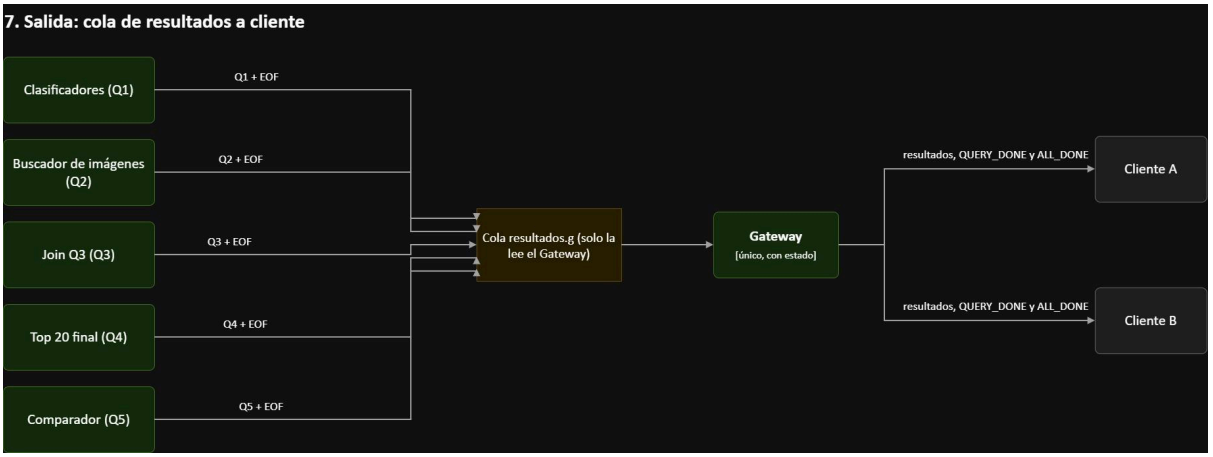
**Rama de comparación:** los artículos desde 2021 llegan por las colas comparacion.i, repartidos por round-robin. El Comparador (con estado, N instancias) guarda url y cantidad en disco hasta que llega el promedio; entonces publica en la cola de resultados los que lo superan y compara cada artículo nuevo al llegar. Manda el EOF de Q5 cuando tiene el promedio, terminó de recorrer el archivo y completó los EOF y conteos de todos los Text processor.



## 7. Salida: cola de resultados a cliente

Todos los clientes se conectan al mismo Gateway, que mantiene sus conexiones TCP. Por eso hay una sola cola de resultados, resultados.g, que solo él lee, y el Gateway identifica a cada cliente por su client_id.

Los cinco nodos que emiten resultados (Clasificador para Q1, Buscador de imágenes para Q2, Join Q3, Top 20 final y Comparador) publican directo en la cola resultados.g, donde g es el gateway_id que viaja en todos los mensajes. Cada mensaje lleva el client_id, el número de consulta y un batch de resultados, y al terminar cada nodo manda el EOF de su consulta. El Gateway busca la conexión del client_id, le manda el batch al cliente y recién entonces hace ack. Además cuenta los EOF de cada consulta, le manda QUERY_DONE al cliente cuando termina cada una y ALL_DONE cuando terminaron las 5. El Cliente separa los resultados por consulta y los escribe en un archivo por consulta a medida que llegan.



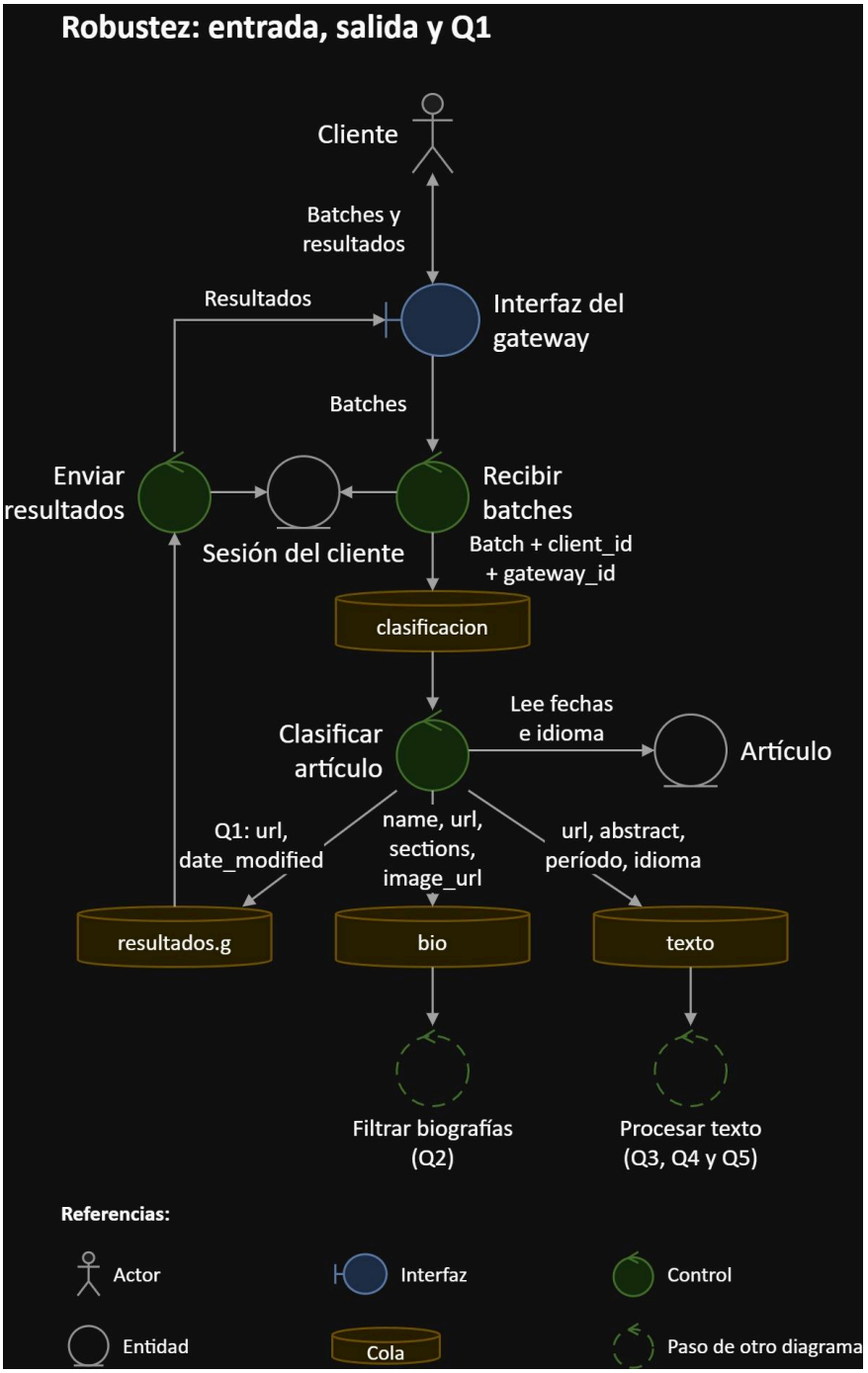
# Diagramas de robustez

El primer diagrama muestra el tramo común de entrada y salida. Los demás arrancan en Clasificar artículo y terminan en Enviar resultados, que aparecen punteados porque se detallan en el primero.

## 1. Entrada, salida y Q1

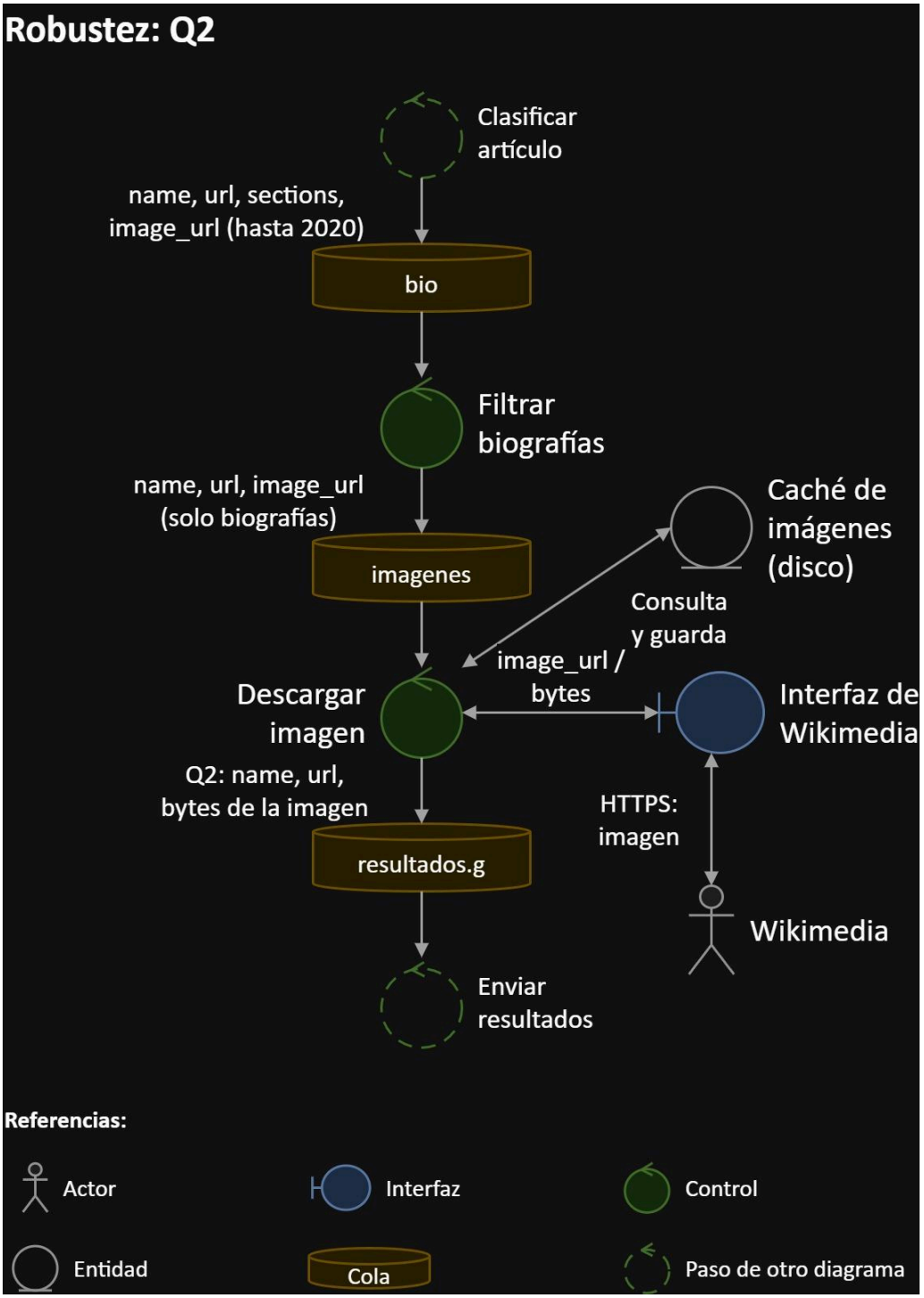
El Cliente manda los batches por la interfaz del gateway. Recibir batches registra la sesión del cliente, les agrega el client_id y el gateway_id y los deja en clasificacion.

Clasificar artículo lee las fechas y el idioma de cada artículo y lo reparte: manda a bio lo que necesita Q2 y a texto lo que necesitan Q3, Q4 y Q5. Además resuelve Q1, porque publica la url y la date_modified de los artículos en inglés hasta 2020 en resultados.g. Enviar resultados toma los resultados de esa cola, busca la sesión del cliente y se los manda por la interfaz del gateway.



## 2. Q2

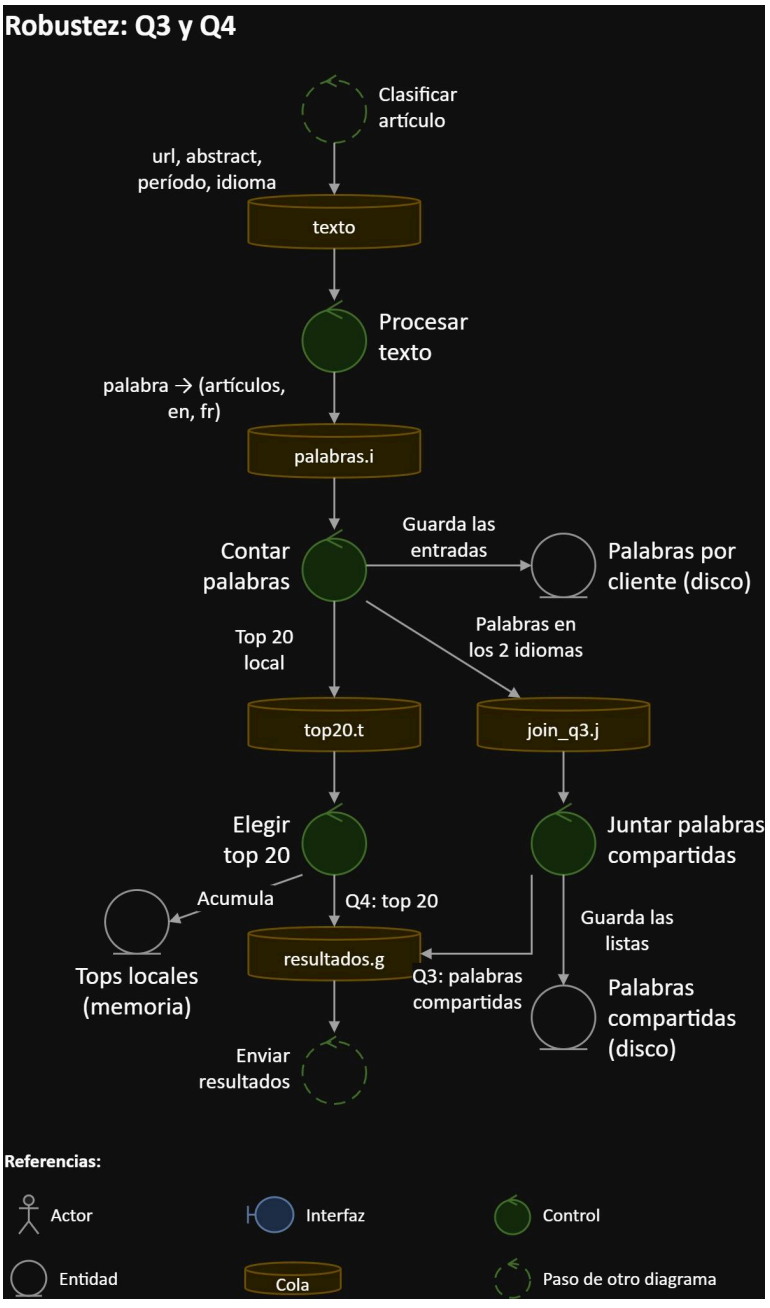
Filtrar biografías recibe de bio los artículos hasta 2020 y deja pasar a imagenes los que mencionan biography o biographie, con su name, url e image_url. Descargar imagen busca cada imagen en la caché y, si no está, la pide a Wikimedia por HTTPS, a través de su interfaz. Reintenta sin límite los timeouts, errores de conexión, HTTP 429 y 5xx, y descarta las imágenes que fallan por otros motivos. Publica en resultados.g el name, la url y los bytes de la imagen.



## 3. Q3 y Q4

Procesar texto saca las palabras únicas de cada abstract, sin stopwords, y las publica en palabras.i agrupadas por palabra, con la cantidad de artículos y los idiomas en los que aparece. Contar palabras guarda esas entradas en disco, en varios archivos por cliente, hasta completar los EOF y los conteos de mensajes. Después manda su top 20 local a top20.t y las palabras que aparecen en los dos idiomas a join_q3.j.

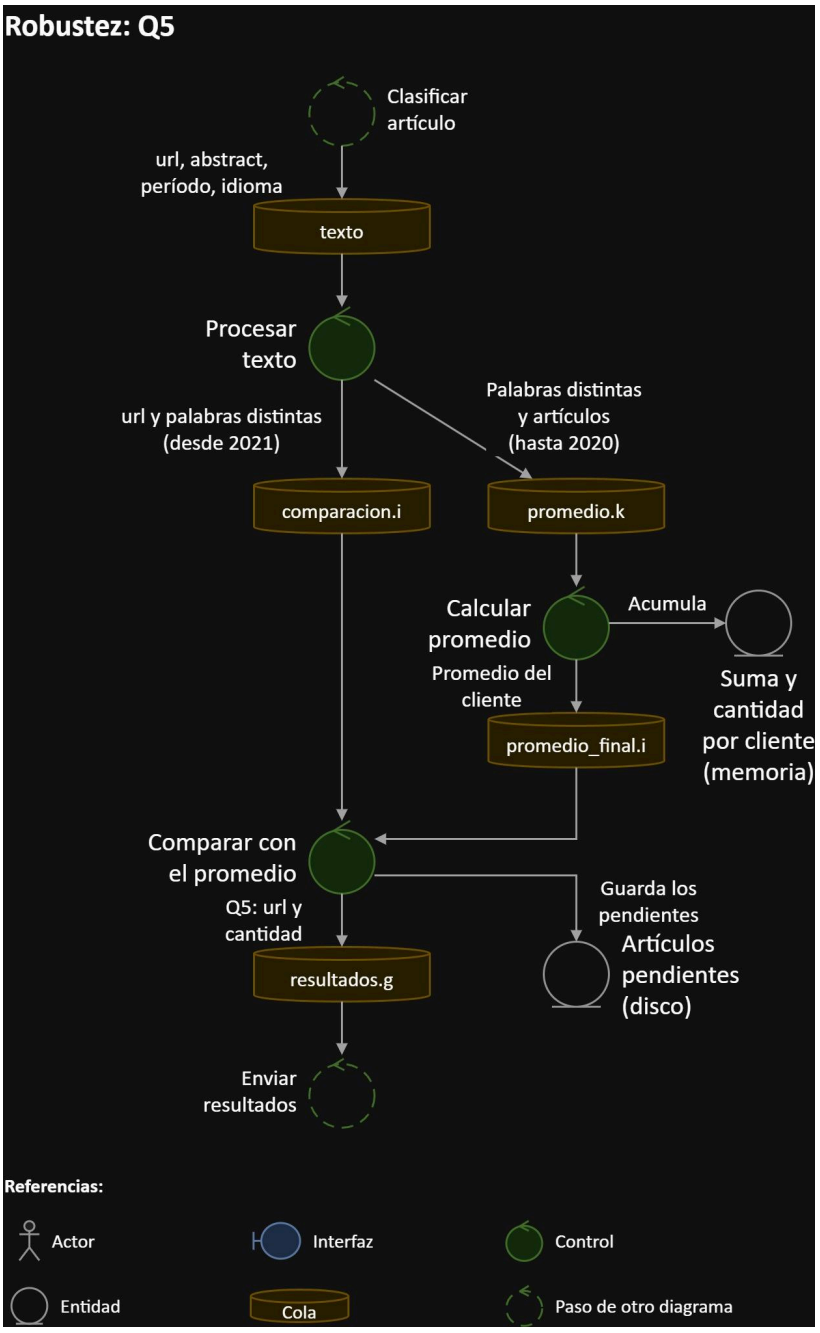
Elegir top 20 junta los tops locales en memoria y publica el top 20 de Q4. Juntar palabras compartidas guarda en disco las listas de todos los contadores y publica las palabras de Q3. Los dos resultados van a resultados.g.



## 4. Q5

Procesar texto manda a promedio.k la suma de palabras distintas y la cantidad de artículos hasta 2020, y a comparacion.i la url y la cantidad de palabras distintas de cada artículo desde 2021. Calcular promedio acumula en memoria la suma y la cantidad del cliente y, cuando terminan sus datos, publica el promedio en promedio_final.i.

Comparar con el promedio guarda en disco los artículos pendientes hasta que llega el promedio. Después publica en resultados.g la url y la cantidad de palabras de los que lo superan, y compara cada artículo nuevo al llegar.

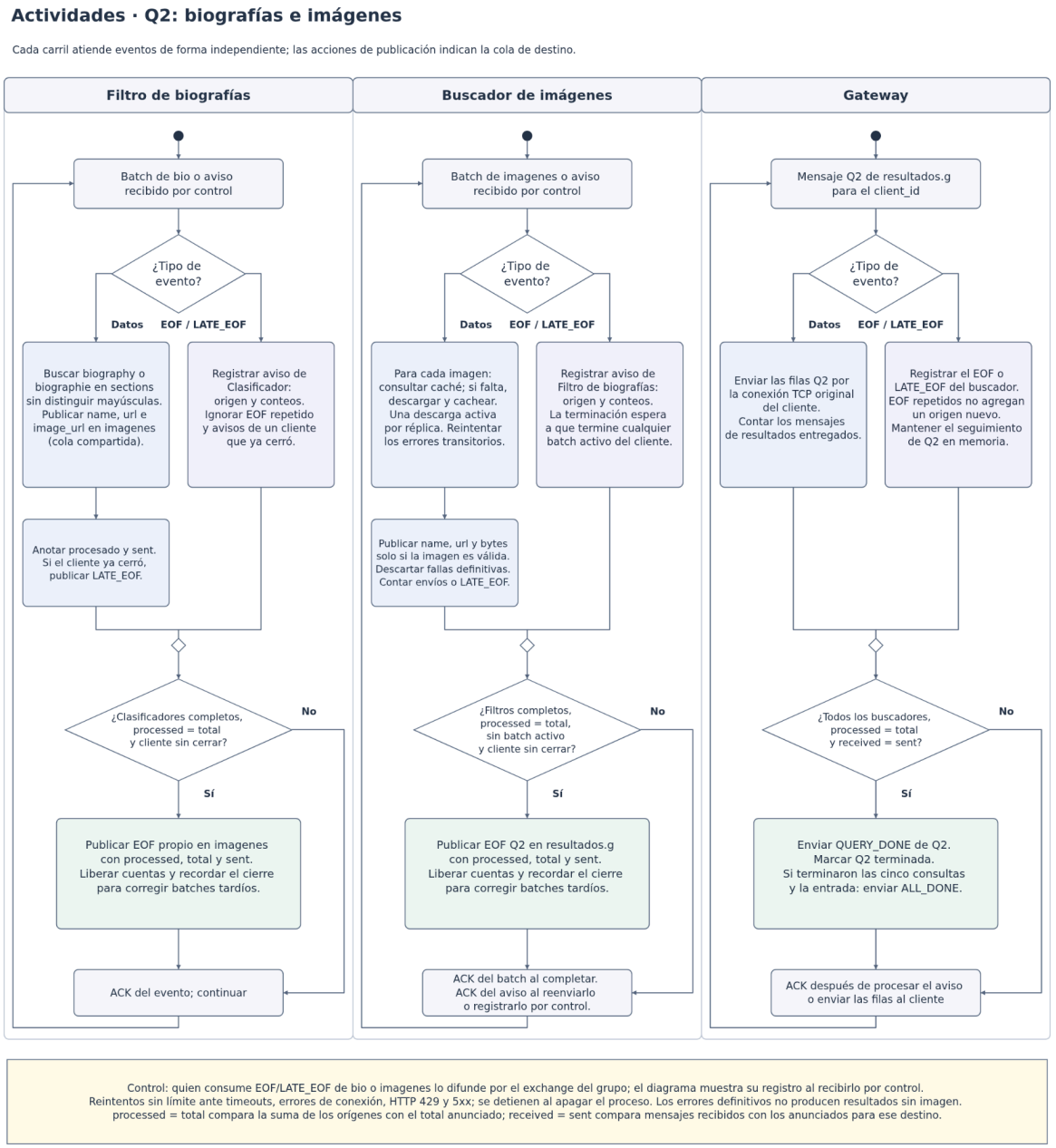


# Diagramas de actividades

Muestran el paso a paso dentro de los nodos: qué hace cada uno con los mensajes que recibe y qué hace cuando llega el EOF de un cliente.

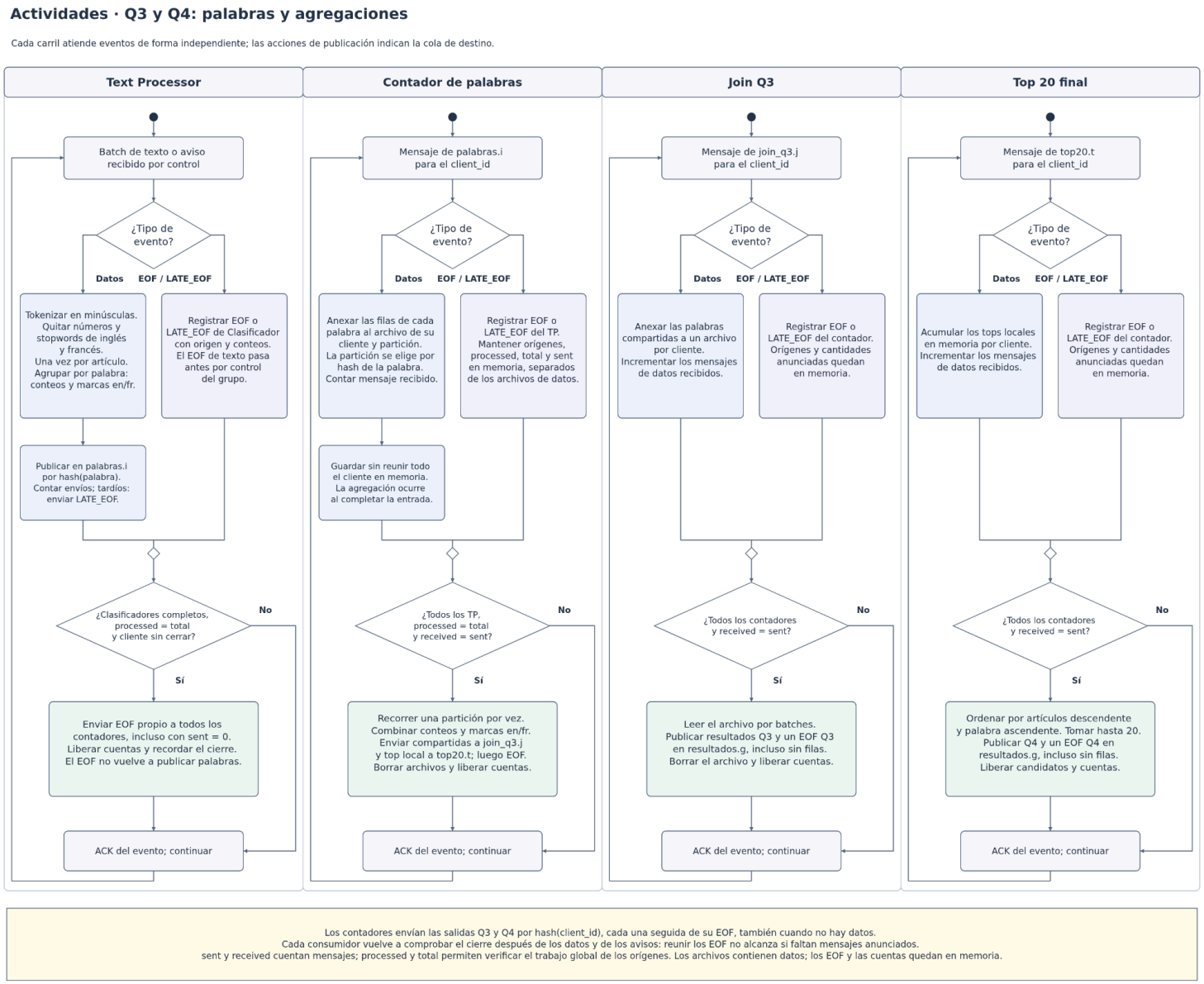
# 1. Q2

Filtrado de biografías y descarga de sus imágenes son actividades procesadas en simultaneo. Los artículos filtrados se reparten entre los buscadores, que publican los resultados de Q2 para el gateway del cliente.



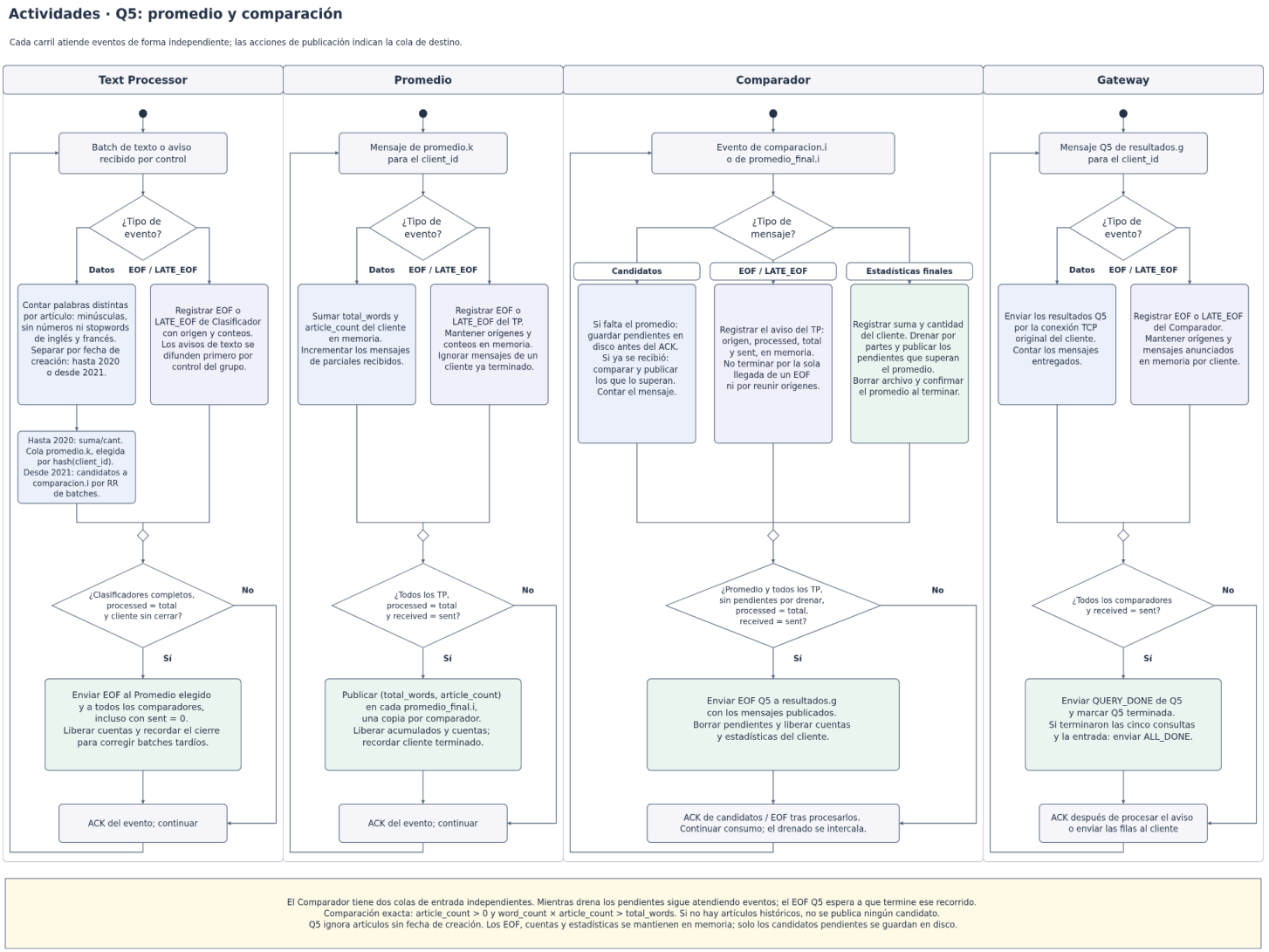
## 2. Q3 y Q4

Text Processor obtiene las palabras sin stopwords y reparte sus conteos por hash de la palabra. Cada Contador de Palabras guarda en disco las entradas de su parte hasta que terminan los datos del cliente.



## 3. Q5

El Text Processor envía los acumulados hasta 2020 al Promedio y los artículos desde 2021 al Comparador. Muestra cómo el Comparador guarda los artículos pendientes o publica los que superan el promedio.



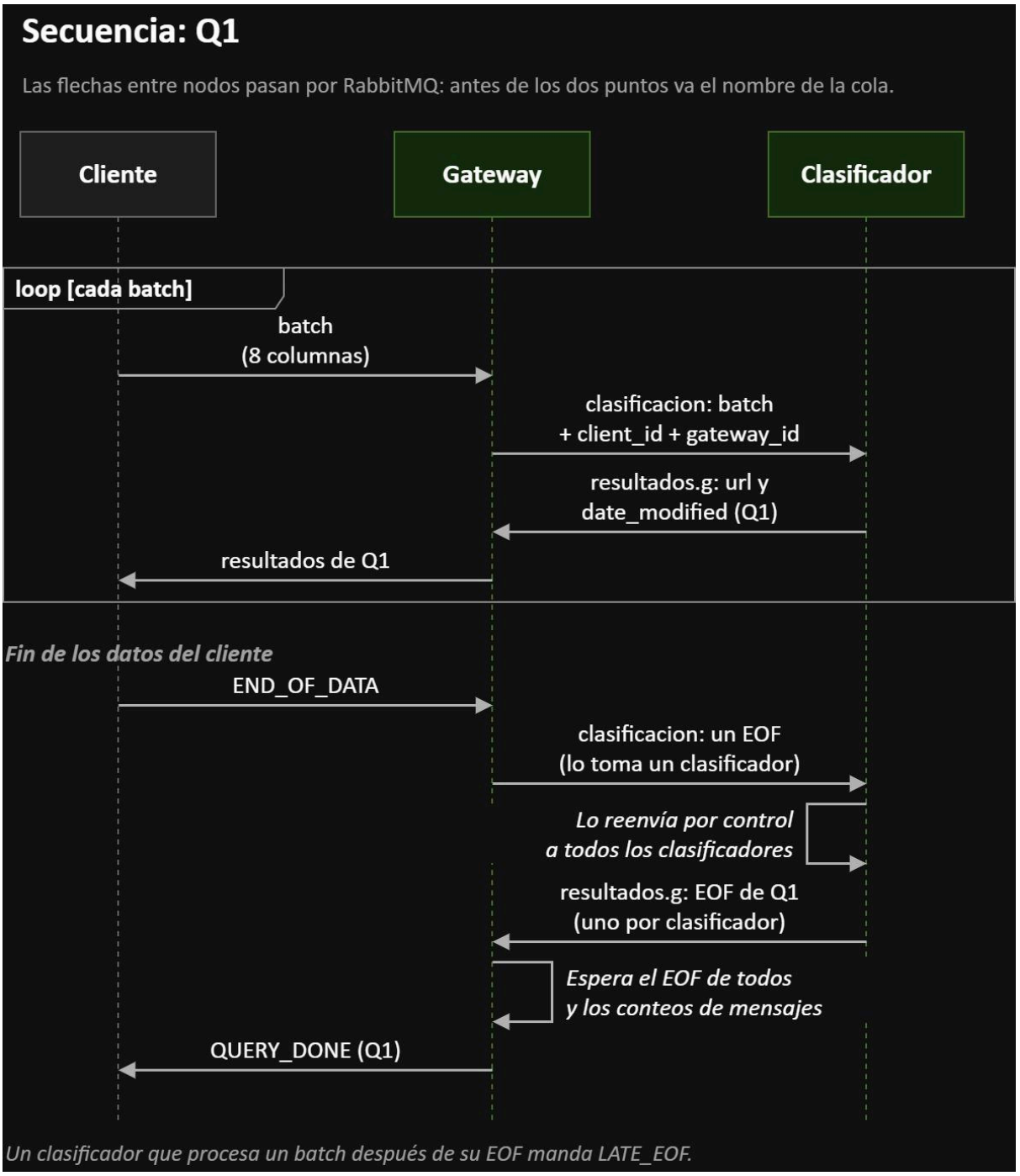
# Diagramas de secuencia

Muestran el orden de los mensajes de Q1, Q2, Q3, Q4 y Q5. Cada participante es una línea de vida, y las flechas entre nodos pasan por RabbitMQ: antes de los dos puntos va el nombre de la cola. El bloque loop muestra lo que pasa con cada batch, y la parte final muestra cómo se propaga el EOF hasta que el gateway le manda al cliente QUERY_DONE. Cuando terminaron las 5 consultas, le manda ALL_DONE.

## 1. Q1

Por cada batch, el Cliente se lo manda al Gateway, que lo publica en clasificacion. El Clasificador resuelve Q1 y publica en resultados.g la url y la date_modified de los artículos en inglés hasta 2020, y el Gateway se las reenvía al Cliente.

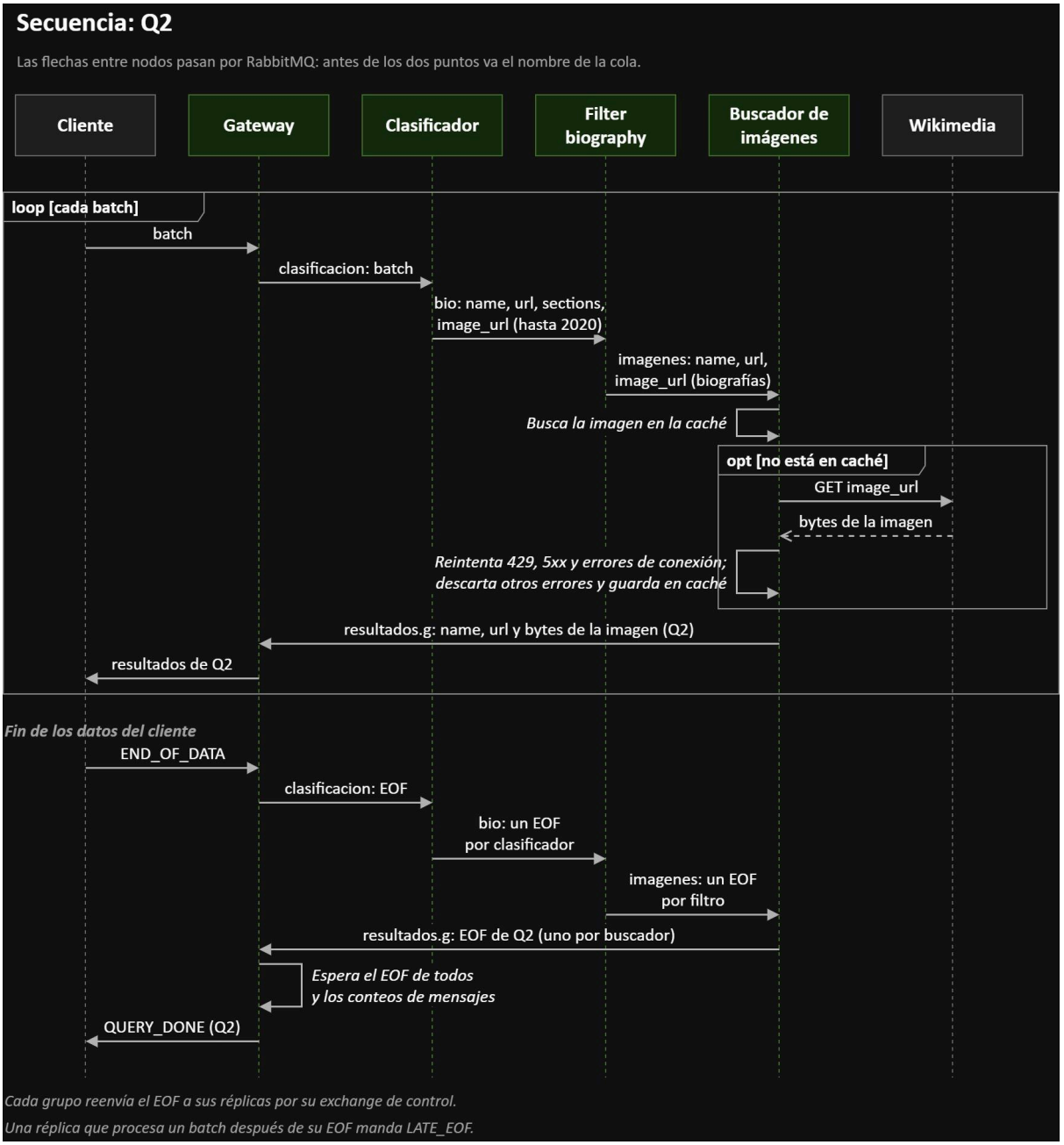
Cuando el Cliente termina, manda END_OF_DATA. El Gateway publica un solo EOF en clasificacion, y el clasificador que lo toma lo reenvía por el exchange de control, así le llega una copia a cada clasificador. Cada uno responde con el EOF de Q1, y cuando el Gateway tiene el de todos y cerraron los conteos de mensajes le manda QUERY_DONE al Cliente.



## 2. Q2

Por cada batch, el Clasificador manda a bio los artículos hasta 2020. El Filter biography deja pasar a imagenes los que mencionan biography o biographie, y el Buscador de imágenes busca cada imagen en la caché y, si no está, se la pide a Wikimedia. Reintenta los timeouts, errores de conexión, HTTP 429 y 5xx, y descarta las imágenes que fallan por otros motivos. Publica en resultados.g el name, la url y los bytes.

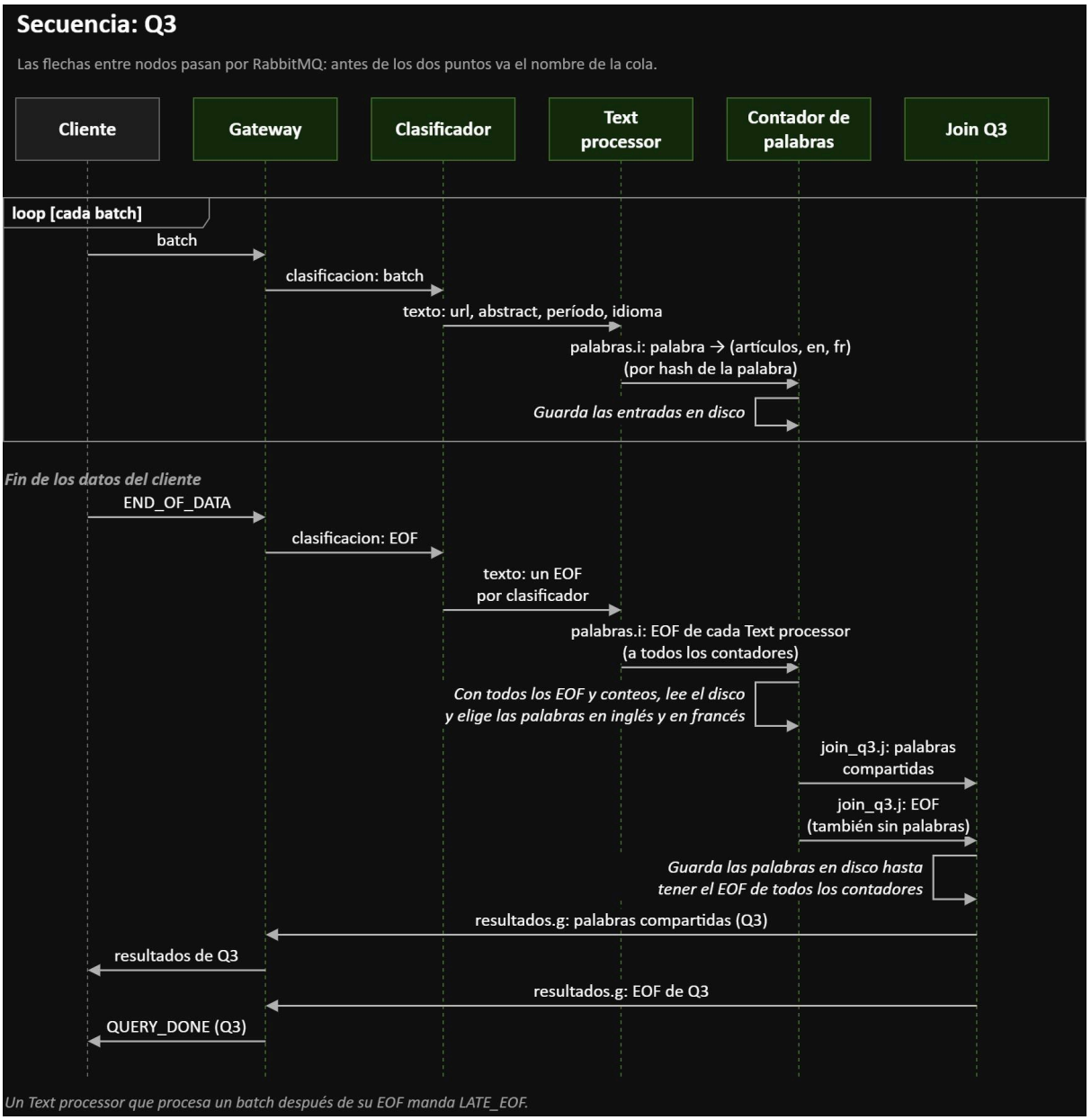
El EOF recorre el mismo camino: Clasificador, Filter biography y Buscador de imágenes. Cada grupo lo reenvía a sus réplicas por su exchange de control, y cada réplica manda el suyo a la etapa siguiente. El Gateway manda QUERY_DONE de Q2 cuando tiene el EOF de todos los buscadores y cerraron los conteos.



## 3. Q3

'Text processor' representa todas sus instancias. 'Contador de palabras' agrupa los shards. 'Join Q3', 'Gateway' y 'Cliente' son las instancias asociadas a ese cliente.

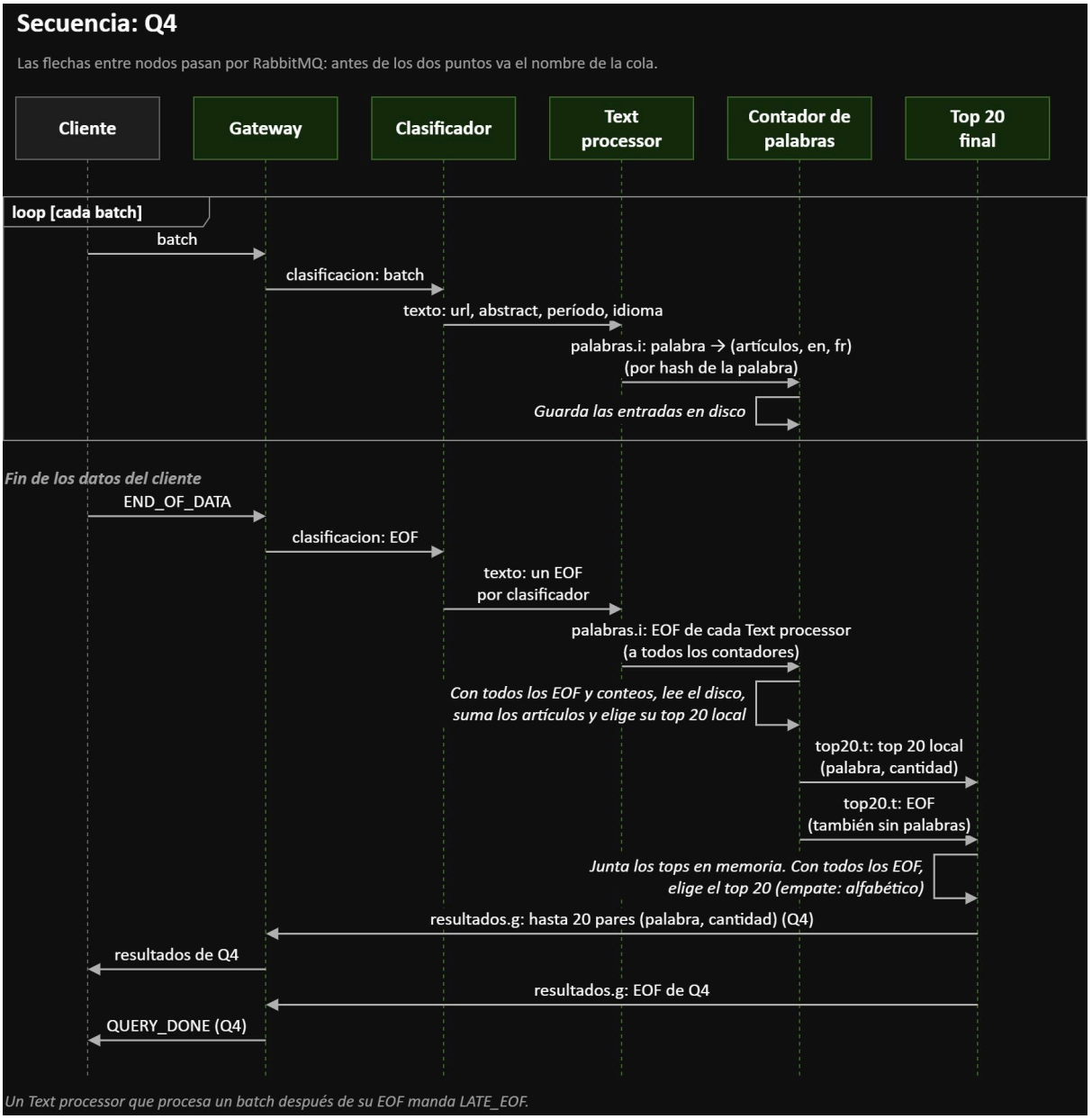
Se hace un hash por palabra para que un solo shard sea responsable de contar esa palabra. Cada contador marca qué palabras aparecen en ambos idiomas. 'Join Q3' guarda las palabras en disco y las publica al recibir el EOF de todos los shards. Los que no recibieron palabras de ese cliente mandan un EOF igual. 'Gateway' reenvía los resultados a medida que llegan y manda QUERY_DONE con el EOF de Q3.



## 4. Q4

Los batches siguen el mismo camino que en Q3 hasta el Contador de palabras, que suma en cuántos artículos aparece cada palabra de su shard y lo guarda en disco.

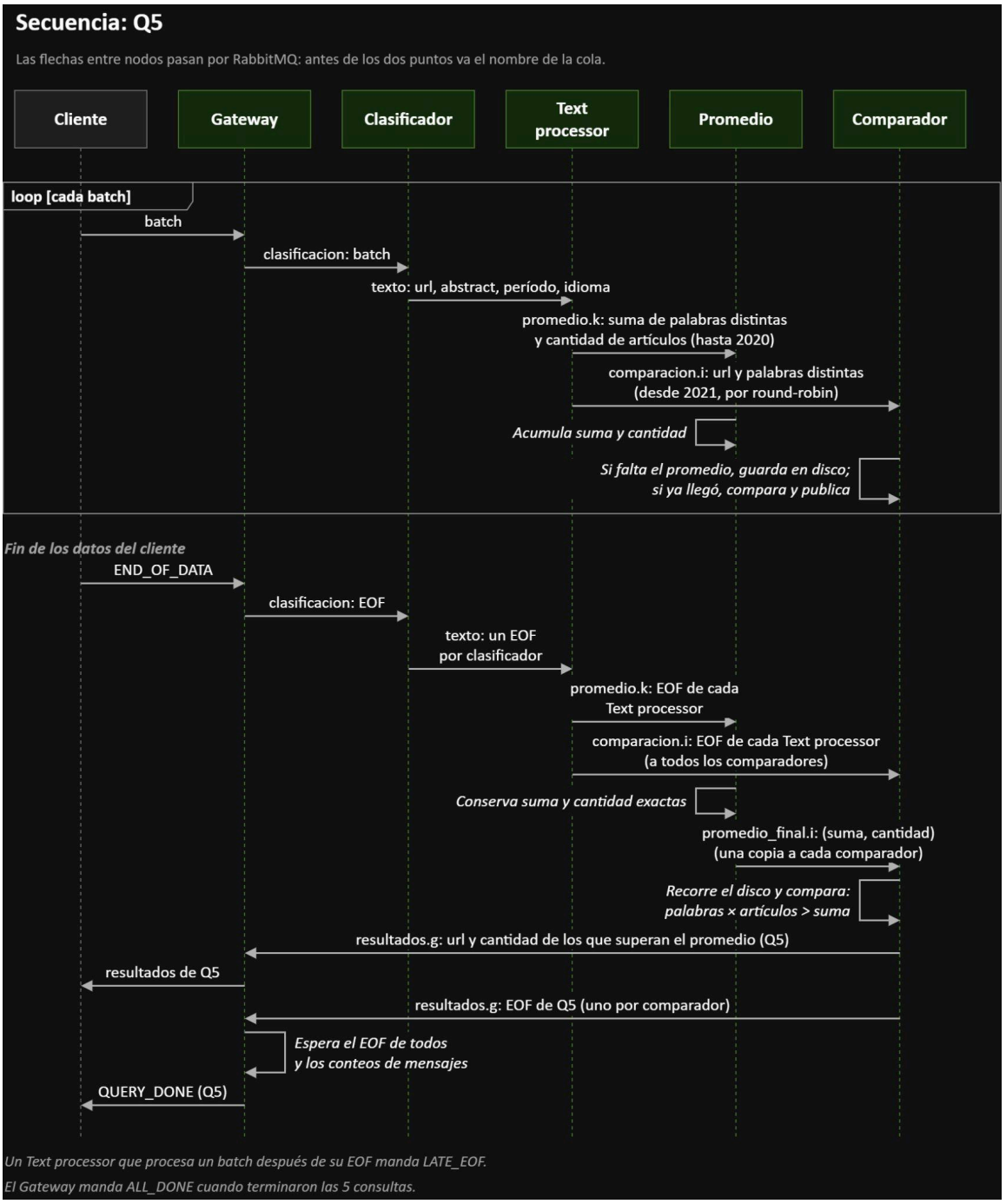
Con todos los EOF y conteos, cada Contador calcula su top 20 local y lo publica en top20.t, seguido de su EOF. El Top 20 final junta los tops locales y, cuando tiene el EOF de todos los contadores, publica en resultados.g las 20 palabras que aparecen en más artículos y manda un solo EOF de Q4. Con ese EOF el Gateway manda QUERY_DONE de Q4.



## 5. Q5

Por cada batch, el Text processor manda al Promedio la suma de palabras distintas y la cantidad de artículos hasta 2020, y el Promedio las acumula. Al Comparador le manda por round-robin la url y la cantidad de palabras distintas de cada artículo desde 2021. Si todavía no llegó el promedio, el Comparador las guarda en disco; si ya llegó, compara y publica.

Con todos los EOF, el Promedio manda la suma y la cantidad exactas a promedio_final.i, una copia por comparador. Cada Comparador recorre el disco, publica en resultados.g los artículos que superan el promedio y manda el EOF de Q5. El Gateway reenvía los resultados a medida que llegan y manda QUERY_DONE de Q5 cuando tiene el EOF de todos los comparadores y cerraron los conteos.



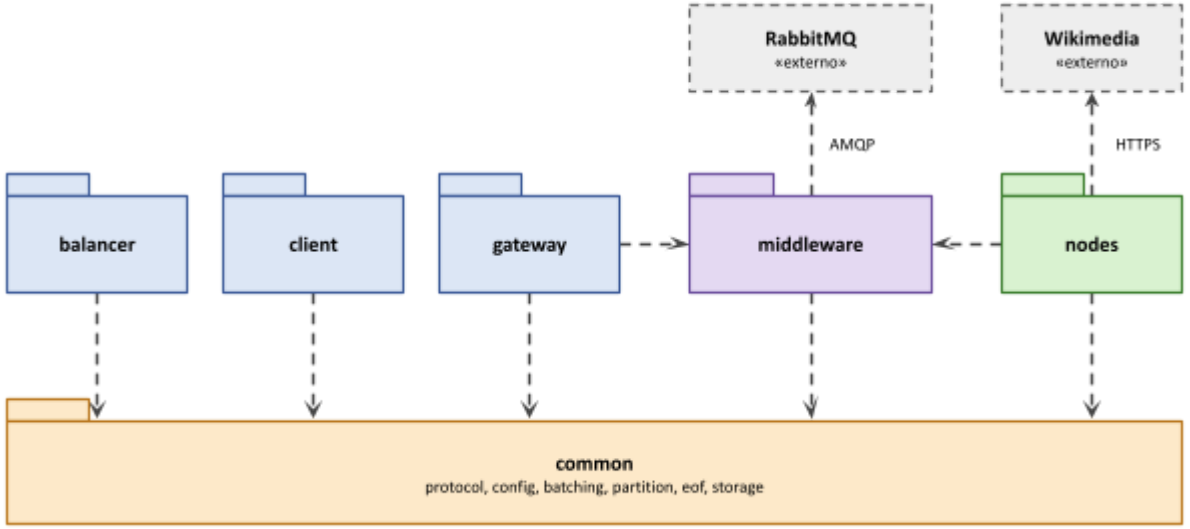
# Diagramas de paquetes

Muestran la vista de desarrollo: cómo se organiza el código en paquetes y qué paquetes dependen de cuáles. Una flecha punteada de A hacia B indica que A usa a B.

Los nodos del sistema no se comunican por dependencias de código sino por RabbitMQ (ver DAG y C4), por eso ningún nodo depende de otro. Los nombres de los paquetes siguen los de los contenedores del C4.

## 1. Vista general

Hay seis paquetes propios y dos sistemas externos. **client**, **balancer**, **gateway** y **nodes** son los programas del sistema, los mismos contenedores del C4. **middleware** es el único paquete que conoce a **RabbitMQ** y lo usan **gateway** y **nodes**. **common** reúne lo que usan todos: los mensajes, la configuración, el armado de batches, el reparto por hash, el conteo de EOF y el acceso a disco. Dentro de nodes, solo **image_searcher** depende de Wikimedia.



## 2. Paquete nodes

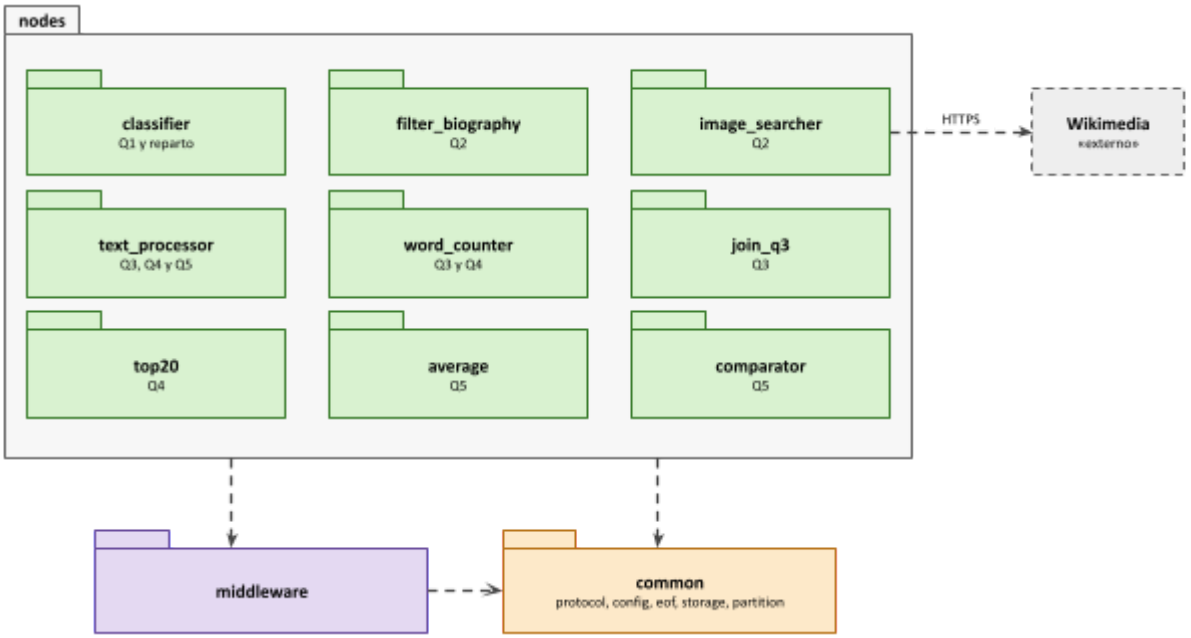
Cada nodo es un subpaquete y ninguno depende de otro: se comunican solo por colas. Los nueve usan middleware para consumir y publicar, y common para los mensajes y los EOF.

Los nodos con estado (**word_counter**, **join_q3**, **top20**, **average** y **comparator**) usan además la parte de storage de common para guardar en disco un archivo por cliente.

Se ordenan por consulta: **classifier** resuelve **Q1**.

**filter_biography** e **image_searcher** resuelven **Q2**.

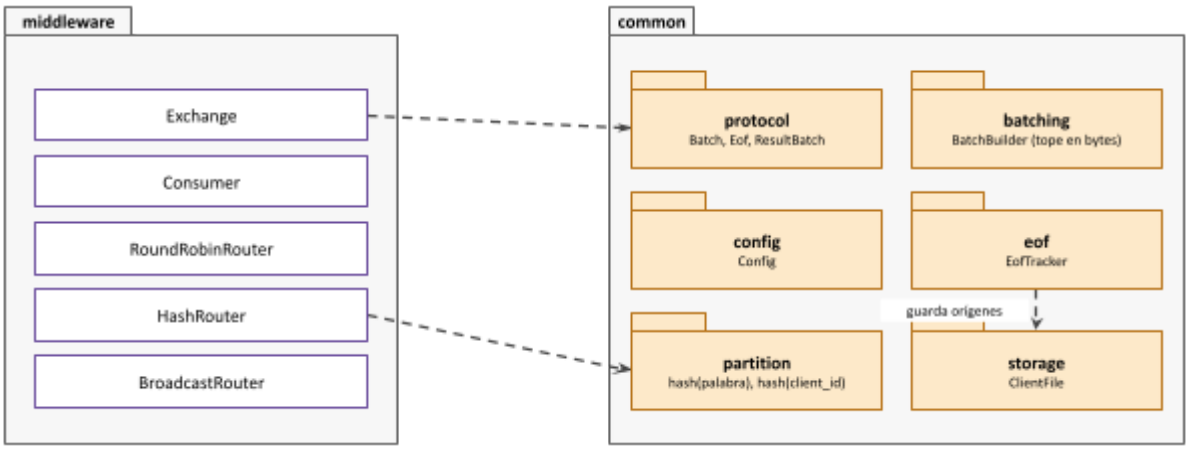
**text_processor** es la etapa común de **Q3, Q4 y Q5**, y el resto completa **Q3, Q4 y Q5**.



## 3. Paquetes middleware y common

middleware ofrece Exchange (direct) y Consumer, con tres formas de conectar colas: WorkQueue (una cola que comparten todas las instancias de un grupo), HashRouter y BroadcastRouter. HashRouter usa partition para elegir la instancia a partir de la palabra o del client_id, y BroadcastRouter se usa para promedio_final y para las colas de control del EOF.

common se divide en seis subpaquetes: **protocol** (los mensajes Batch, Eof y ResultBatch, con client_id, gateway_id y número de consulta), **config** (la cantidad de instancias de cada etapa, de dónde sale, cuantos EOF espera cada nodo), **batching** (arma batches con un tope en bytes), **partition** (los hashes), **eof (cuenta los orígenes de los EOF, no los mensajes, y en los grupos con cola compartida reenvía el EOF por el exchange de control)** y **storage** (un archivo por cliente, eof guarda ahí sus orígenes).



# Diagramas de despliegue

Muestran dónde corre cada parte del sistema: el Cliente, el Balanceador, los gateways, RabbitMQ y las instancias de cada nodo, que nodos se comunican entre si y qué nodos necesitan disco para guardar su estado mientras esperan el EOF de un cliente.

